

INVARIA for Evidence Based Application Security Using Invariant Guided Graph Analysis

Shashank Kakad¹ Soham Moholkar Omkar Kakade Yash Pardeshi

¹Vishwakarma Institute of Technology
Pune, India

Corresponding author: shashank.kakad24@vit.edu

Abstract— Application security weaknesses can arise when an API accepts a valid request but fails to preserve the ownership, input, or business constraints expected by the application. Detecting these weaknesses requires understanding how request data reaches an operation and which controls are present along that path. This paper presents INVARIA, a local application security system that combines source parsing, a typed security graph, invariant-guided decision rules, and optional language-model review. Its INVARIANT engine requires rule-specific source evidence before reporting an authorization, injection, credential, or sensitive business-flow candidate. A Python and FastAPI backend coordinates analysis, while a Next.js dashboard presents the graph, evidence, coverage, and human review decisions. PostgreSQL with vector retrieval supports contextual guidance, and SQLite provides a smaller local configuration. Evaluation on three vulnerable/fixed fixture pairs correctly reported all three vulnerable cases and suppressed findings in their corrected counterparts. Thirty repeated analyses per fixture produced median engine times between 0.054 and 0.708 milliseconds, excluding ingestion, persistence, and inference. An archived application scan resolved 19 Express routes and produced 16 authorization candidates requiring review. These observations support the feasibility of transparent local security triage within a bounded analysis scope, while broader vulnerability accuracy and production scalability remain to be evaluated.

Keywords— Application Security, Static Analysis, Security Invariants, Authorization, Security Graphs, Evidence Verification

I] INTRODUCTION

The increasing dependence on web applications has made application security an essential requirement for organisations, educational institutions, and online services. Modern applications expose functions through APIs, connect to databases, and divide processing among controllers, services, and client interfaces. These arrangements improve development flexibility, but they also distribute security decisions across several parts of the source code. An endpoint may receive a user identifier, invoke a helper function, and update a private record without making the ownership condition visible in one place. Consequently, examining an isolated line is often insufficient to understand whether the operation follows the intended security policy.

Application weaknesses are not limited to visibly dangerous instructions. A database lookup can use a legitimate query while returning another user's information. A refund endpoint can perform a valid update while accepting an amount chosen by the client. Similarly, an application can load a configuration value from the

environment but fall back to a predictable credential when the deployment omits that value. These situations demonstrate the need to examine relationships between input, identity, data access, and expected controls. The important question is whether an operation preserves the conditions required by the application rather than whether its syntax is unusual.

Traditional static analysis provides useful detection of recognisable patterns, including unsafe query construction and embedded credentials. However, developers also need to understand why a particular pattern was reported and what assumptions affect the decision. A warning that does not show the request source, affected operation, or ownership context can be difficult to investigate. Large collections of findings increase the review effort when developers cannot distinguish a directly observed weakness from a condition that depends on external configuration. For small teams, this investigation cost can reduce the practical value of an otherwise useful security tool.

Language models offer another way to explain code and discuss security intent. Their generated explanations can be helpful when an application contains several connected functions or when the expected business behaviour is described outside the code. Nevertheless, a fluent explanation does not establish that the cited operation exists or that an attack is possible. A model may misunderstand a framework convention, overlook an external control, or refer to an unsupported source location. Security decisions therefore need an explicit relationship between the model's interpretation and the evidence that the system actually extracted.

INVARIA addresses this problem through a local pipeline in which deterministic analysis establishes the candidate and optional model review provides additional interpretation. The central policy is INVARIANT, expanded as Invariant-based Networked Verification and Attack-path Reasoning. An invariant represents a condition expected to remain true during a security-sensitive operation, such as restricting an order lookup to the authenticated owner. The system searches for request-controlled operations and checks whether supported source patterns establish the corresponding control. When the required evidence is incomplete, generated prose cannot supply the missing proof.

The application accepts a public GitHub repository or a bundled fixture, retains an isolated source snapshot, and extracts routes and functions from supported JavaScript and TypeScript files. It builds a typed graph that associates endpoints with handlers, queries, data models, and observed security boundaries. Candidate findings preserve source snippets, file locations, unresolved assumptions, and the

result of the evidence gate. A dashboard enables developers to examine these details, confirm or dismiss a finding, and export the resulting report. This organisation makes the investigation process a visible part of the system.

The objective of this research is to describe and evaluate a practical method for evidence-based security triage using the existing INVARIA implementation. The work examines whether the system distinguishes vulnerable fixtures from corrected variants, whether bounded helper analysis preserves useful provenance, and whether the local analysis engine operates with low overhead on small inputs. It also examines a previously recorded application scan to illustrate route coverage and candidate reporting. These questions concern the behaviour of a working prototype rather than a claim that all application security conditions can be verified statically.

The contribution is the integration of source-derived hypotheses, rule-specific evidence gates, and a persistent decision ledger within a lightweight review workflow. This design gives a developer access to the observed path and the conditions that remain unresolved. It also makes the limits of a finding explicit: source verification establishes provenance, while exploitability still depends on the application's policy and runtime environment. The following sections discuss related work, the methodology and implementation, measured results, future improvements, and the conclusions that can be drawn from the current evaluation.

III LITERATURE REVIEW

Yamaguchi, Golde, Arp, and Rieck [1] introduced code property graphs for representing program structure and discovering vulnerabilities. Their representation brings abstract syntax, control flow, and program dependence into a combined graph that can be queried for vulnerable patterns. This work provides a foundation for considering security as a relationship among program elements. INVARIA follows the general motivation of making relationships inspectable, but its current graph is a narrower application-oriented representation. It connects supported routes, functions, lookups, and models; it does not implement the complete control-flow and dependence representation evaluated in that research.

The OWASP API Security Top 10 discussion of Broken Object Level Authorization [2] identifies the importance of checking access to an object selected through an API. This risk is relevant when a request supplies an identifier that reaches a private lookup without an appropriate access condition. In INVARIA, this becomes a source-analysis hypothesis: a request-derived selector reaches a data operation and no supported server-derived owner or tenant predicate is established. This translation is intentionally conditional because source inspection may not reveal database policies or later access checks that protect the same operation.

OWASP also describes unrestricted access to sensitive business flows [3], emphasising that legitimate functionality may still be abused when an application fails to restrict how it is used. This provides context for examining payments, inventory changes, and other high-impact operations.

INVARIA's business-flow rule covers a narrower situation: client-controlled sensitive fields reaching a mutation without recognised control markers. It does not implement every form of business-flow abuse, including automated excessive use. Repository declarations help state expected controls, but detecting their names is not equivalent to verifying the complete business process.

Semgrep's taint-analysis documentation [4] explains how sources, sinks, sanitizers, and propagators support rules that follow unsafe data. Such analysis is useful for identifying input that reaches an executable operation. INVARIA includes a separate Semgrep Community Edition baseline in its Linux deployment and also implements bounded local request propagation. The baseline retains its own provenance rather than being presented as a first-party route-to-sink trace. Tree-sitter [5] supplies the parsing infrastructure used by the prototype, enabling structured extraction from JavaScript and TypeScript instead of relying only on textual matching.

Lewis et al. [6] presented retrieval-augmented generation as a method for combining model knowledge with retrieved external information. INVARIA adapts this general idea to the retrieval of curated security guidance before optional model review. The retrieved material supports explanations and remediation, while the repository supplies the evidence for a finding. Liu et al. [7] investigated how language models use long contexts and observed sensitivity to the position of relevant information. This motivates compact evidence bundles in INVARIA, although that study does not measure the security accuracy of this application or its configured model.

The Common Weakness Enumeration entries for SQL injection [8] and hard-coded credentials [9] provide established definitions for two additional candidate classes. Together, these sources motivate structural input analysis, explicit ownership reasoning, and careful handling of generated explanations. The implementation examined here combines these ideas into one review workflow rather than proposing a new parsing technique or language model. Its research question is whether the integration produces inspectable and repeatable decisions on supported examples, with a clear distinction between a source-backed candidate and a confirmed vulnerability.

III METHODOLOGY / EXPERIMENTAL

This section describes the analysis procedure, system components, evaluation inputs, and measurement protocol. INVARIA was examined as a local application security prototype, with the Express route analysis serving as its principal path. The evaluation used the current working source tree [10] on 3 October 2026 and retained historical reports separately. Deterministic fixture measurements excluded network access and model execution. This separation allows the reported engine times to be interpreted without confusing them with repository acquisition, database transactions, or the duration of an optional language-model review.

III.I] Algorithm for Invariant Guided Security Analysis

The INVARIANT algorithm begins with a hypothesis about a supported security-sensitive operation. For an authorization candidate, it asks whether request-controlled data selects a private object and whether the same lookup establishes an owner or tenant condition using server-derived identity. For injection, it asks whether request data reaches executable query, command, or code text. For a business-flow candidate, it asks whether client-controlled values reach sensitive fields without the declared controls. Credential analysis examines literal assignments and recognised configuration defaults. Each hypothesis is associated with a minimum set of evidence kinds.

The decision procedure operates in the following stages. First, eligible files are collected into a bounded source snapshot. Second, parsing extracts routes, handlers, imports, and supported data operations. Third, the engine associates request inputs and identity values with direct operations or resolved local helper calls. Fourth, it checks the supported safe conditions and proposes a candidate where the hypothesis remains applicable. Fifth, the evidence gate requires the appropriate route, operation, or literal evidence. Finally, source verification checks that the retained locations and snippets correspond to the input before the finding is included in the report.

For the principal rules, BOLA001 requires route and lookup evidence; BFL001 requires route and mutation evidence; INJ001 requires route and sink evidence; and SEC001 requires credential-assignment evidence. A resolved cross-function path additionally records flow evidence at the caller. These requirements make the reported decision reproducible from the extracted material. They do not prove every semantic assumption about that material. In particular, an ownership predicate that the engine recognises is a supported static control pattern, while an omitted external control remains an explicit question for a reviewer.

The decision ledger stores the hypothesis, required and observed evidence kinds, pass or discard result, recognised controls, risk signals, and unresolved conditions. It also records whether a bounded interprocedural trace was established. Confidence values are fixed rule heuristics rather than calibrated probabilities of exploitation. The implementation increases confidence slightly when a supported helper trace improves provenance, but this adjustment does not turn a candidate into a runtime demonstration. Preserving these distinctions is necessary when the output is used for prioritisation or compared across scans.

III.II] System Overview and Architecture

The backend uses Python and FastAPI to coordinate ingestion, parsing, analysis, persistence, and report retrieval. A single local worker processes a bounded queue and records pipeline stages. SQLAlchemy manages scan storage, with SQLite available for the smaller workstation setup and PostgreSQL available for the richer deployment. The immutable report, graph, evidence, model audit, and human feedback are retained together for each scan. An interrupted scan is marked as failed on startup; automatic crash recovery

and distributed execution are outside the current worker design.

The frontend uses Next.js, React, and TypeScript to expose the scan and review workflow. A developer selects a fixture or repository, examines the discovered endpoints, and opens candidates to inspect supporting snippets and assumptions. The interface also presents coverage so that unresolved handlers or unexamined source areas are visible. Review actions persist a confirmation or dismissal with the scan record. JSON and Markdown exports retain the underlying analysis and review information, allowing the findings to be discussed outside the interface without depending on a transient dashboard session.

The architecture can be summarised as repository snapshot, structured parsing, typed security graph, invariant-guided hypotheses, evidence verification, optional contextual model review, and human assessment. Each stage has a different responsibility. Parsing establishes structure, detector rules propose a security concern, and the gate determines whether the minimum evidence exists. Retrieval supplies guidance rather than program facts. The model explains the supplied evidence, while a human resolves application-specific assumptions. This division helps identify where a failure occurred and prevents an explanation from silently replacing the analysis result.

III.III] Experimental Setup and Testing Procedure

The controlled dataset consists of six bundled fixtures arranged into three vulnerable/fixed pairs. The authorization pair contrasts an object lookup selected by request input with a lookup scoped to the authenticated identity. The injection pair contrasts executable SQL constructed from request input with a corrected parameterized form. The credential pair contrasts an embedded credential-like literal with a corrected environment-based configuration. Each vulnerable fixture has an expected candidate count of one, while each fixed fixture has an expected count of zero. These expectations are deliberately small and directly inspectable.

The fixture evaluation invokes the same analysis entry point used by the application. An initial untimed run warms each fixture, after which thirty sequential calls are measured using Python's high-resolution performance counter. File inventory is prepared before measurement. Each timed call includes parsing, graph construction, built-in detection, and source-evidence verification on the supplied in-memory files. The median and the nearest-rank 95th percentile are calculated from the thirty observations. The experiment runs under Python 3.11.9 on the Windows workstation and reports elapsed time rather than isolated CPU execution time.

The regression suite provides additional checks of local helper propagation, ownership-scoped suppression, sensitive payment mutations, declared control markers, unsafe repository URL rejection, document extraction, model-reference rejection, and the scan/review/export lifecycle. It was run separately from the timing experiment. The live PostgreSQL integration test is conditional on database configuration and was skipped in this run. Consequently, the current regression result does not represent a fresh verification of live vector retrieval. A retained

PostgreSQL/model report provides historical evidence of that path without being merged into the current test count.

For application-scale illustration, the study uses the archived TaskForge report [11] generated on 8 September 2026. TaskForge is a scan target rather than INVARIA's own codebase. The report identifies repository commit 4360fa0e5f6dd9ff9f636680dafd57031324f8c4 and records coverage, candidate counts, retrieval mode, model status, and total pipeline duration. Because the current source tree contains later changes, the archived run is treated as a separate case study. It is not used to estimate the present engine's recall or to claim that all recorded candidates were independently confirmed vulnerabilities.

III.IV] Software and Tools Used

Tree-sitter and its JavaScript and TypeScript grammars extract structured program elements from the supported files. The route resolver recognises static Express declarations, basic named handlers, relative imports, and simple router mounts. The graph is stored as typed JSON nodes and edges rather than requiring a separate graph database. This keeps the prototype's installation smaller while still supporting relationships that a reviewer can inspect. The implementation pins the parser version and places native analysis in a subprocess within the application pipeline to bound failures and processing time.

PostgreSQL with pgvector stores the configured 768-dimensional guidance embeddings and supports cosine-distance retrieval. The local SQLite configuration uses an explicitly labelled category-based guidance fallback. Ollama exposes the optional Gemma review model and EmbeddingGemma embedding model through configurable provider settings. The system does not require a remote paid inference service. If the reasoning model is missing or returns invalid output, the result remains a deterministic candidate with the model status reported separately. Availability of the optional model therefore does not determine whether the static engine can run.

Semgrep Community Edition supplies an additional deterministic baseline in the Linux Docker configuration. Its results are verified against source evidence and retain their separate origin. The current Windows fixture timing excludes Semgrep execution. This is important because a deployment containing an external scanner has different startup and processing costs from an in-memory engine call. The prototype also includes limited supplemental checks for Python credential defaults and anonymous Firebase client data operations. These checks do not establish Python request-flow coverage or verify Firebase security rules, and they are outside the six-fixture timing dataset.

III.V] Source Graph and Detailed Decision Steps

Consider a route that accepts an order identifier and invokes a service function with that identifier and the authenticated user identity. The parser resolves the import where supported, and the analysis propagates those arguments into the helper's parameters. A lookup using only the order identifier supports an authorization hypothesis, so the finding retains route, caller-flow, and lookup evidence. If

the helper instead constrains the same lookup by both identifier and server-derived user identity, the recognised scope condition suppresses the candidate. Regression tests exercise both sides of this example.

The mechanism is bounded rather than a complete whole-program taint analysis. Ambiguous function names are not guessed, and handler following stops at the existing resolver bound. Supported local aliases can preserve request provenance, but arbitrary framework behaviour, complex exports, and full control-flow dominance are not established. An authorization check performed after a query can require reasoning beyond the recognised lookup predicate. Similarly, a database row policy may protect a record without appearing in the application source. These limitations are retained in the report so that a reviewer can examine the unresolved environment.

Business-flow analysis considers request-controlled changes to high-impact fields such as amount, balance, refund, price, inventory, and status. A repository-local invariant declaration can identify a route, relevant fields, and expected control markers. If any required marker is absent, the candidate records the missing names. If every declared marker is present, the generic candidate can be suppressed within the supported policy. This confirms marker coverage only. A function called withTransaction may exist without correctly enclosing the mutation, and a function called requireIdempotency may implement an insufficient check.

The detailed procedure therefore preserves a useful distinction between syntactic control presence and semantic correctness. Static source evidence can show that input reaches a mutation and that a named control is referenced. It cannot by itself establish all allowed state transitions, transaction outcomes, refund ceilings, or concurrent execution behaviour. For such cases, INVARIA identifies the operation and the missing or recognised markers as review material. The application policy, runtime tests, and developer assessment remain necessary to determine whether the business invariant is actually satisfied.

III.VI] Evidence Review and Contextual Reasoning

Optional model review receives a compact bundle containing the original candidate, verified source evidence, unresolved assumptions, and relevant guidance. Its response must follow a structured schema, and evidence identifiers must refer to the candidate's supplied evidence. Explicit file-and-line citations are also checked against supported locations. An invented evidence identifier or unsupported source location causes validation failure. These checks provide a mechanical boundary around references, although they do not prove that every sentence in the model's explanation is a correct interpretation of the code.

Business documentation can be attached to explain intended ownership, workflow, and API behaviour. Uploaded documents are marked as untrusted context and stored separately from the repository snapshot. They are not parsed as application code, inserted into the graph, or accepted as evidence for a finding. Their role is to help frame a question such as whether a job is private to its creator or whether a refund requires approval. Instructions contained in

a repository or uploaded document cannot satisfy the source-evidence gate simply by asserting that an operation is safe.

The reviewer receives the original analysis alongside the advisory explanation. Confirmation and dismissal therefore concern an inspectable candidate rather than an unexplained model verdict. A dismissal can reflect an external control that the static engine does not model; a confirmation can establish the expected policy and missing check. Persisting these decisions makes it possible to retain application context for later assessment. The current implementation records feedback but does not use it to claim automatic learning, retrain the reasoning model, or generate a validated patch without further work.

III.VII] Performance Considerations and Resource Bounds

The smallest deployment performs static analysis with a local database and no model download. Source ingestion excludes dependencies, generated build output, binaries, and symlinks, and imposes limits of 2,000 eligible files, 500 KB per file, and 20 MB total. Native parsing runs with a two-minute timeout; the Linux configuration additionally applies an address-space bound. These measures keep one repository from consuming unbounded resources. They also mean that a large application can have incomplete coverage, which must be considered when interpreting a result with few or no candidates.

The engine uses supported syntax and bounded local relationships to keep the initial analysis practical. Optional inference adds a different resource profile, including model memory and sequential review time. A single worker simplifies local operation but limits throughput when several scans are queued. The dashboard and report store stage information so that a developer can distinguish ingestion, analysis, and review. No throughput, memory, energy, or infrastructure-cost benchmark was collected in this study, so the timing results establish only the behaviour of the measured fixture analysis path.

IV] RESULTS AND DISCUSSIONS

The evaluation combines current controlled measurements with a separately identified historical application scan. The six fixtures test whether the principal rules produce the expected candidate counts after a known code correction. The timing experiment measures repeated engine calls on these small inputs. The regression suite tests additional behavioural boundaries, while the TaskForge archive illustrates how coverage and candidate reporting appear on a larger target. These observations are complementary, but they do not share the same input scale, execution path, or implementation date and should not be treated as interchangeable benchmarks.

IV.I] Vulnerable and Corrected Fixture Results

All six fixture cases produced their expected outputs. The vulnerable authorization case returned one BOLA001 candidate, the vulnerable injection case returned one INJ001 candidate, and the vulnerable credential case returned one

SEC001 candidate. Their fixed counterparts returned zero findings. Figure 1 presents this separation by category. The corrected cases are important because a detector that reports every example would identify the vulnerable cases without demonstrating sensitivity to the relevant control. Here, recognised ownership scoping, parameterized values, and environment-based credential loading changed the result as expected.

For the predefined binary fixture labels, the observations correspond to three true positives, three true negatives, zero false positives, and zero false negatives. Precision, recall, and accuracy are consequently 100 percent within this six-case set. These values describe fixture agreement, not a general vulnerability-detection rate. The examples are hand-authored, each contains one expected condition, and they do not sample the diversity of production frameworks or controls. The thirty repeated calls per case check repeatability and timing; they do not increase the number of independent labelled programs beyond six.

Vulnerable and corrected fixtures

Observed candidates per fixture | 6 independent cases

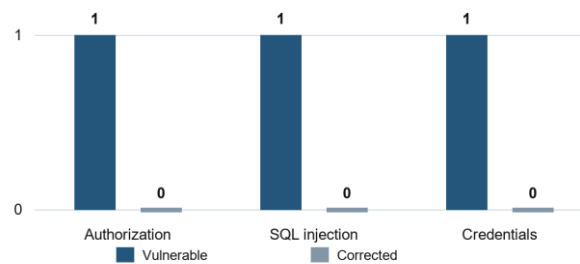


Fig. 1. Candidate counts in the three vulnerable/fixed pairs. Current evaluation, 3 October 2026.

The regression run completed with 41 tests passed and one optional PostgreSQL integration test skipped. Relevant checks confirmed that a resolved helper path retains flow evidence, an identity-scoped helper lookup suppresses the authorization candidate, and incomplete business-control declarations retain the sensitive mutation candidate. Tests also rejected fabricated model evidence references and unsupported source locations. Passing these checks demonstrates the intended implementation behaviour for their specific inputs. It does not establish that a generated explanation is always correct or that the engine recognises every valid ownership or business-control pattern.

IV.II] Analysis Time and Application Coverage

Figure 2 shows median elapsed engine times across thirty measured repetitions for each fixture. The authorization fixed and vulnerable cases recorded medians of 0.708 and 0.416 milliseconds, respectively. The injection fixed and vulnerable cases recorded 0.229 and 0.318 milliseconds. The credential fixed and vulnerable cases recorded 0.054 and 0.080 milliseconds. The corresponding 95th-percentile values were 0.783, 0.453, 0.258, 0.355, 0.065, and 0.092 milliseconds. All observations concern already inventoried, in-memory fixture sources and exclude model inference and the surrounding application pipeline.

Static engine analysis time

Median and p95 | 30 warned calls per fixture

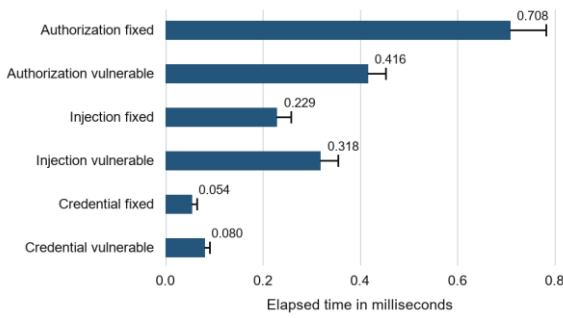


Fig. 2. Median engine time with p95 whiskers. In-memory analysis only; ingestion, storage and model review excluded.

The fixed authorization fixture took longer than its vulnerable counterpart in this run. A correction does not necessarily make a scan faster, since the engine still has to parse the additional ownership condition and evaluate whether it establishes a supported safe pattern. Differences also reflect fixture content and workstation scheduling. No statistical inference about performance superiority is drawn from this ordering. The result shows that the measured small-input path completed below one millisecond at the median for each case; it does not establish the same latency for a repository scan initiated through the dashboard.

The archived TaskForge report records six ingested source files, one file counted as parsed, 19 discovered routes, 19 resolved routes, and 15 functions. Figure 3 compares discovered and resolved endpoint counts with the 16 reported authorization candidates. Route resolution covered all endpoints discovered by that run, but this is not equivalent to analysing every file or proving complete attack-surface coverage. The graph and candidate evidence make the discovered operations inspectable, while the coverage counters expose the difference between the ingested inventory and the parser's recorded examination of that inventory.

Archived TaskForge scan coverage

8 September 2026 | Candidates require human review

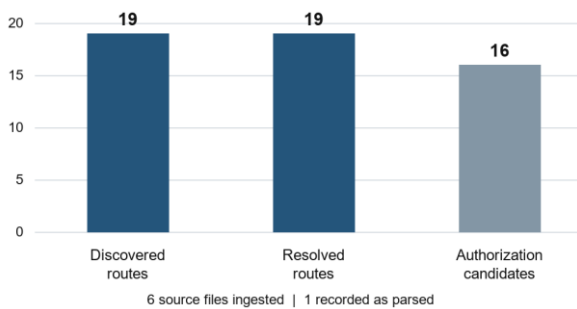


Fig. 3. Discovered routes, resolved routes and candidates from the archived TaskForge report. Counts represent different measures.

The 16 candidates were labelled high severity by the recorded rules, and the archive shows accepted structured model responses for all 16, with pgvector guidance retrieval and completed model status. The recorded total pipeline duration was 268.64 seconds. Acceptance indicates that the structured response and references passed the review

interface's validation; it does not mean that 16 vulnerabilities were independently confirmed. The duration includes a different workflow from the fixture experiment. It therefore provides an example of optional review overhead without supporting a direct speed comparison or a model-attributed accuracy improvement.

IV.III] Comparison and Interpretation

The prototype provides a different kind of output from a warning that identifies only a local code pattern. Each supported candidate contains an operation, related evidence, and a ledger describing the conditions required for reporting it. Compared conceptually with unrestricted model review, the decision gate provides a reproducible minimum-evidence boundary. These are architectural differences rather than results from a controlled head-to-head benchmark. This study did not run competing scanners under a shared configuration, independently label a large corpus, or evaluate an ablation with and without retrieval or model review.

The observations support three limited conclusions. The predefined vulnerable/fixed pairs are separated correctly, bounded helper and policy behaviours pass their regression checks, and the small-input engine path has low measured elapsed time on this workstation. The application archive additionally demonstrates recorded route discovery and review output. Generalisation remains restricted by the small dataset, the older case-study implementation, and the absence of independent exploit confirmation. False-positive and false-negative rates on realistic repositories require external labels, explicit coverage criteria, and repeatable scans using the same implementation version.

These limitations are especially important for business logic. A named transaction or approval helper can be present while the underlying operation remains unsafe, and a valid external control can be absent from the scanned source. Consequently, a zero-finding report is not a security certificate. INVARIA is useful as a way to organise investigation and preserve the reasoning behind a candidate. The current results justify further evaluation of that workflow, with particular attention to unsupported code paths and whether reviewers can reach correct decisions more efficiently using the evidence ledger.

V] FUTURE SCOPE

V.I] Broader Invariant Verification

The present business-flow rule identifies sensitive mutations and declared control markers. Future work should connect these observations to stronger verification of application invariants. For a refund, this could include checking that the amount does not exceed the captured payment, that the caller is authorised, and that repeated requests cannot create repeated credits. Such verification requires understanding allowed state transitions and the placement of checks relative to writes. Adding control-flow analysis and explicit invariant models would help distinguish the presence of a named control from evidence that the operation actually obeys it.

Concurrency and transaction boundaries need particular attention. An operation may pass a balance check and still be unsafe when simultaneous requests change the same value. A complete analysis must consider whether validation and mutation are atomic and whether the persistent state supports the expected idempotency rule. Static reasoning can identify likely boundaries, but targeted runtime tests may be needed to validate the final claim. A future workflow should preserve the separate provenance of static observations and dynamic results rather than presenting an inferred attack path as an executed exploit.

V.III] Larger and Independent Evaluation

A broader evaluation should use independently labelled applications covering multiple coding styles, data layers, and supported security controls. Vulnerable/fixed pairs remain useful, but they should include difficult negative cases such as post-query checks, database row policies, unusual tenant predicates, and framework-specific identity propagation. Metrics should distinguish unsupported cases from analysed negatives and report precision and recall by category. This would give a more informative result than combining all candidate types into one accuracy value, particularly where different rules rely on different kinds of evidence.

Controlled comparisons should also measure the contribution of the graph, evidence gate, retrieval, and optional reasoning stage. The same source snapshots and labels should be used for each configuration, with comparable resource limits and tool versions. A reviewer study could examine whether the ledger reduces investigation time and improves decisions when external controls are involved. Hardware, warm-up, repetition counts, and complete pipeline timings should be recorded so that measured latency can be reproduced. These additions would address the current study's limited scale without overstating the results of its six fixtures.

V.III] Extended Language and Framework Coverage

The main path currently depends on statically declared Express routes and bounded JavaScript or TypeScript relationships. Future versions could support additional routing frameworks, complex module exports, middleware composition, and broader argument propagation. Python support could expand from credential-default checks to explicit route and data-flow analysis. Firebase-related analysis could incorporate separately obtained security rules where authorised. Each addition should have its own coverage report and negative controls, because recognising another file extension is not equivalent to providing reliable security analysis of its runtime behaviour.

V.IV] Durable Processing and Deployment

Moving beyond a trusted workstation requires durable jobs, worker leases, scan-level authorisation, and isolation between users and organisations. A distributed queue would allow interrupted work to resume and several workers to process separate scans safely. Resource limits should remain attached to individual jobs, and source snapshots should be retained with reproducible identifiers. Model execution could

be scheduled separately so that expensive review does not block deterministic scanning. Throughput and failure-recovery experiments would be required before claiming that the current local design scales to a shared production service.

V.V] Improved Review and Remediation

The dashboard could connect dismissed findings to the external controls that justified dismissal and show how a changed snapshot affects the original evidence. Reviewers could compare routes, control predicates, and policy declarations across versions instead of relying only on severity counts. Proposed remediation should eventually be linked to targeted regression tests demonstrating the expected security condition. Automatically generated patches would require compilation, behavioural checks, and human review before use. These improvements would extend the present evidence-centred workflow while retaining a clear separation between detected patterns, advisory explanations, and validated corrections.

VI] CONCLUSION

This paper presented INVARIA as a local application security system that combines structured parsing, a typed security graph, invariant-guided candidate generation, and optional contextual reasoning. Its INVARIANT decision policy requires source evidence appropriate to each rule and preserves the resulting decision in a ledger. The dashboard exposes that evidence together with coverage and review actions, allowing developers to inspect the basis of a candidate and the assumptions that remain unresolved. This provides a practical foundation for application-security investigation within the supported analysis scope.

The controlled evaluation correctly separated all three vulnerable fixtures from their corrected counterparts. Thirty repeated analyses per fixture produced median engine times from 0.054 to 0.708 milliseconds, and the regression suite passed 41 tests with one optional database integration test skipped. A separate archived TaskForge scan recorded 19 resolved routes and 16 authorization candidates with optional model review. These results demonstrate specific implementation behaviours and a working review pipeline; they do not establish comprehensive vulnerability recall, general production latency, or independently confirmed exploitation of the application candidates.

Further work should strengthen semantic invariant verification, broaden framework coverage, and evaluate realistic applications with independent labels. Durable processing and authenticated multi-user isolation are also necessary for shared deployment. With these extensions and appropriate validation, INVARIA can develop into a more capable tool for identifying security-sensitive operations and supporting consistent review. The present contribution is an inspectable, bounded approach in which a security claim remains connected to source evidence and a developer can see both what was observed and what still needs to be established.

References

- [1] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and Discovering Vulnerabilities with Code Property Graphs,” IEEE Symposium on Security and Privacy, 2014. <https://www.ieee-security.org/TC/SP2014/papers/ModelingandDiscoveringVulnerabilitieswithCodePropertyGraphs.pdf>
- [2] OWASP Foundation, “API1 2023 Broken Object Level Authorization,” OWASP API Security Top 10, 2023. <https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>
- [3] OWASP Foundation, “API6 2023 Unrestricted Access to Sensitive Business Flows,” OWASP API Security Top 10, 2023. <https://api-security.owasp.org/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/>
- [4] Semgrep, “Taint Analysis Overview,” official documentation, accessed 3 October 2026. <https://docs.semgrep.dev/writing-rules/data-flow/taint-mode/overview>
- [5] Tree-sitter, “Introduction,” official documentation, accessed 3 October 2026. <https://tree-sitter.github.io/tree-sitter/>
- [6] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, 2020, arXiv 2005.11401. <https://arxiv.org/abs/2005.11401>
- [7] N. F. Liu et al., “Lost in the Middle How Language Models Use Long Contexts,” arXiv 2307.03172, 2023. <https://arxiv.org/abs/2307.03172>
- [8] MITRE, “CWE 89 Improper Neutralization of Special Elements Used in an SQL Command,” Common Weakness Enumeration, accessed 3 October 2026. <https://cwe.mitre.org/data/definitions/89.html>
- [9] MITRE, “CWE 798 Use of Hard-coded Credentials,” Common Weakness Enumeration, accessed 3 October 2026. <https://cwe.mitre.org/data/definitions/798.html>
- [10] INVARIA project, current working implementation and regression fixtures, local research artefact, evaluated 3 October 2026. Source modules: engine.py, algorithm.py, policy.py, reasoning.py, and supplemental_checks.py; six bundled fixtures; regression suite.
- [11] INVARIA project, “TaskForge Final Scan Report,” local archived research artefact, 8 September 2026. Report artifacts/taskforge-final-report.json; target commit 4360fa0e5f6dd9ff9f636680dafd57031324f8c4.